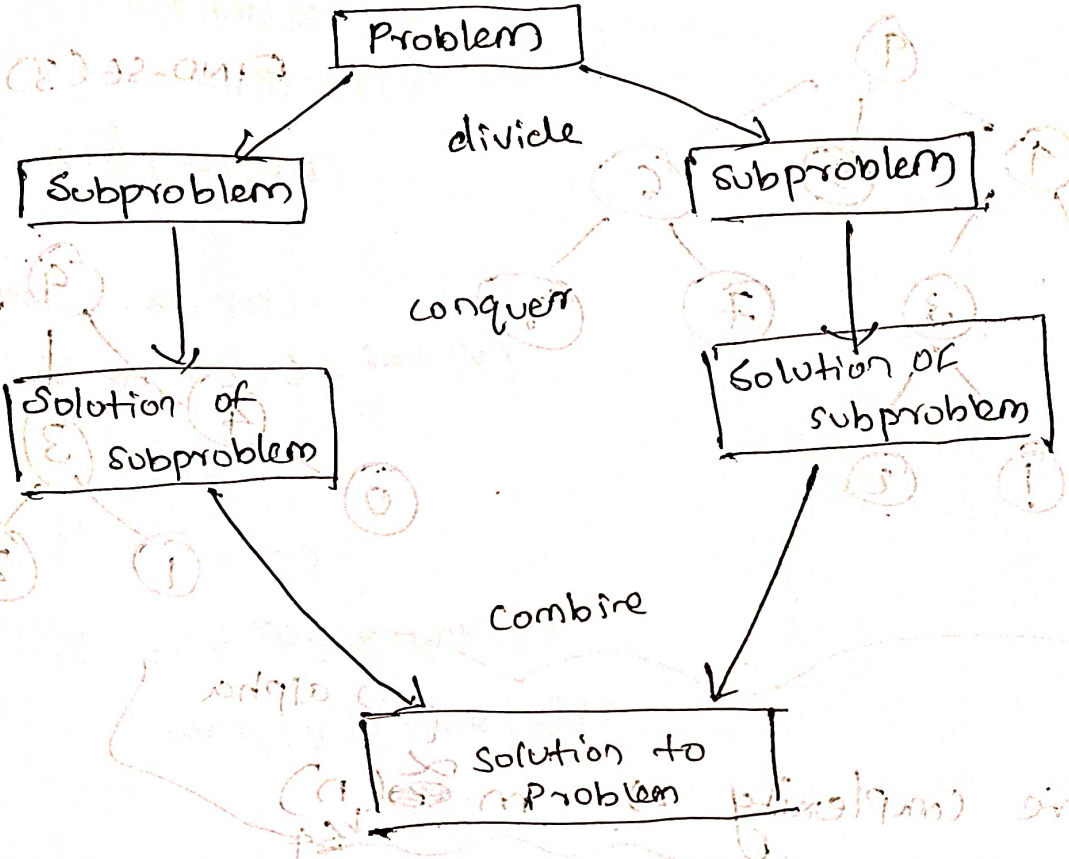
Module-3

Divide and Conquer



* Each and every divide and conquer is to a recurrance relation

P is the problem to be solved

Algorithm DAndC(P)

{

if small(P) then return S(P);

else

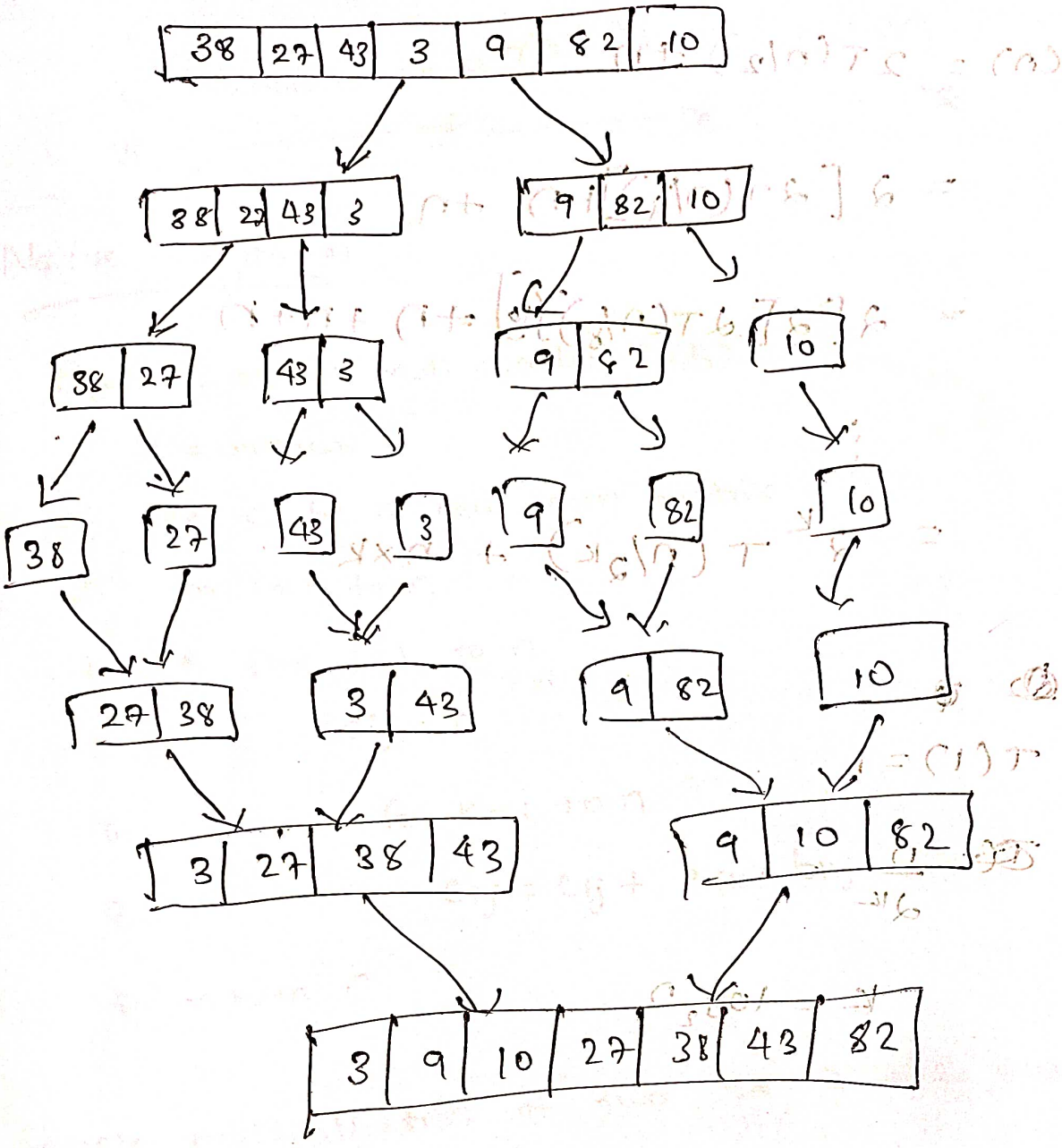
```
{
    divide p into small instances P1, P2 ---- Pk   k ≥ 1
    Apply DandC to each of these subproblem.
    return combine (DandC(P1), DandC(P2) ..... DandC(Pk))
}
}
```

Merge Sort

```
Algorithm MergeSort (low, high)
{
    if (low < high) then
    {
        mid := ⌊(low + high)/2⌋;        // divide
        MergeSort(low, mid);            // conquer
        MergeSort(mid+1, high);
        Merge (low, mid, high);         // combine
    }
}
```

```
Algorithm
    Merge (low, mid, high)
{
    h = low
    i = low
    j = mid + 1
    while (h ≤ mid and j ≤ high) do
    {
        if (a[h] ≤ a[j]) then
        {
            b[i] = a[h];
```

```
        h=h+1;
    }
    else
    {
        b[i] = a[j];
        h=h+1;
    }
    i=i+1
}
if (h>mid) then
    for k=j to high do
    {
        b[i] = a[k];
        i=i+1;
    }
else
    for k=h to mid do
    {
        b[i] = a[k];
        i=i+1;
    }
for k=low to high do a[k]=b[k]
}
```



Time complexity analysis

General form of divide & conquer:

$$T(n) = aT(n/b) + D(n) + C(n)$$

a ⇒ no: of subproblem

n/b ⇒ size

In merge Sort

$a = 2 \quad b = 2$

$$T(n) = 2T(n/2) + n$$

$$= 2[2T(n/4)] + n + n$$

$$= 2[2[2T(n/8)] + n] + n + n + n$$

$$= 2^k T(n/2^k) + n \times k$$

$$T(1) = 1$$

$$\frac{n}{2^k} = 1$$

$$k = \log_2 n$$

$$= 2^{\log_2 n} \times T(1) + n \log_2 n$$

$$= n + n \log_2 n$$

$$= n(1 + \log_2 n)$$

$$O(n \log_2 n)$$

Strassen's Matrix Multiplication

Naiive algorithm

```
Square_matrix Multiplication (A,B)
1. n = arrows
2. let C be a new nxn matrix
3. for i=1 to n
4.     for j=1 to n
5         cij = 0
6         for k=1 to n
7.            cij = cij + aik * bkj
8. return C
```

Sample multiplication of two 2×2 matrices

number of multiplication $= 8 = n^3$

number of addition $= 4 = n^2$

DandC (matrix Multiplication)

divide: each nxn into four $n/2 \times n/2$ matrices

recurrance equation for this approach

$T(n) = 8T(n/2) + n^2$

↓

4 for matrix A
4 for matrix B

$T(n) = 8T(n/2) + n^2$

$= 8(8T(n/4) + (n/2)^2) + n^2$

$= 8^2 T(n/4) + 2n^2 + n^2$

$= 8^2(8T(n/8) + (n/4)^2) + 2n^2 + n^2$

$= 8^3 T(n/8) + 4n^2 + 2n^2 + n^2$

⋮

$= 8^k T(n/2^k) + n^2(1 + 2 + 4 + \cdots + 2^{k-1})$

$= 8^k T(n/2^k) + n^2(2^k - 1)$

$n/2^k = 1$

$k = \log n$

$= 8^{\log_2 n} \times 1 + n^2(2^{\log n} - 1)$

$= n^3 + n^2(n-1)$

$= n^3 + n^3 - n^2$

$O(n^3)$

$P_1 = a(f-h)$ $\quad P_2 = (a+b)h$

$P_3 = (c+d)e$ $\quad P_4 = d(g-e)$

$P_5 = (a+d)(e+h)$ $\quad P_6 = (b-d)(g+h)$

$P_7 = (a-c)(e+f)$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} P_5+P_4+P_2+P_6 & P_1+P_2 \\ P_3+P_4 & P_1+P_5+P_3+P_7 \end{bmatrix}$$

$C_{12} = P_1 + P_2$

$= a(f-h) + (a+b)h$

$= af - ah + ah + bh$

$= af + bh$

Q. multiply using Strassen's matrix multiplication

$$\begin{bmatrix} a & b \\ 1 & 3 \\ c & d \\ 5 & 7 \end{bmatrix} \begin{bmatrix} e & f \\ 8 & 4 \\ g & h \\ 6 & 2 \end{bmatrix}$$

$P_1 = a(f-h) = 2$ $\quad P_2 = (a+b)h = 8$ $\quad P_3 = (c+d)e = 96$

$P_4 = d(g-e) = -14$ $\quad P_5 = (a+d)(e+h) = 80$ $\quad P_7 = (a-c)(e+f) = -48$

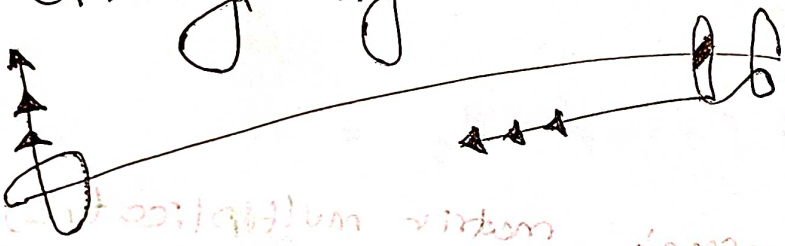
$P_7 = (b-d)(g+h)$

$= -32$

$$\begin{bmatrix} P_5 + P_4 + P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

$$= \begin{bmatrix} 28 & 10 \\ 82 & 34 \end{bmatrix}$$

Greedy Algorithms



* An advanced algorithm design strategy
* used for optimization problems
* simple & straightward
* greed algorithm always makes the choice that look best at the moment ⇒ local optimum
* It works in stages.
* Short sighted
* never reconsiders this decision.

5 pillars

1) A candidate set, from which a solution is created
2) A selection function, which chooses the best candidate to be added to the solution.
3) A feasibility function, ie used to determine if a candidate can be used to contribute to a solution.
4) An objective function, which assigns a value to a solution or a partial solution
5) Solution function (optimal solution), which will indicate when we have discovered a complete solution.

Control abstraction

```
Algorithm Greedy (a,n)
{ solution = ϕ   // initialize the solution
  for i = 1 to n do
  {
      x = select (a);
      if feasible (solution, x) then
      {
          solution = union (solution, x);
      }
  }
  Return solution;
}
```

Greedy choice property

A globally optimum solution can be arrived

Knapsack problem

2 types

→ 1. 0-1 knapsack problem

* either to take an item fully or to leave it

* It is cannot be efficiently solved by greedy algorithm.

→ 2. Fractional knapsack problem

* can take the fraction of items

* can be solved easily by greedy.

Fractional knapsack problem

Formally the problem can be stated as

Maximize $\sum p_i$

subject to $\sum w_i x_i \leq m$

And $0 \leq x_i$

Algorithm

m ⇒ capacity

n ⇒ no: of items

```
greed knapsack(m, n)
{
// Assume that items are sorted in decreasing order of
   profit per unit
```

for i = 1 to n do x[i] = 0

U = m

~~for i = 1 to n do~~

{

for i = 1 to n do

{

if

1.

Item	A	B	C	D
Profit	24	18	18	10
weight	24	10	10	7

	A	B	C	D
P/W	1	1.8	1.8	1.43

Decreasing order B C D A

initial $n = 4$ $m = 25$

$S = \{ 0, 0, 0, 0 \}$

B

Profit = 18

$wt = 25 - 10 = 15$

$S = \{ 0, 1, 0, 0 \}$

C

Profit = 18 + 18 = 36

$wt = 15 - 10 = 5$

$S = \{ 0, 1, 1, 0 \}$

D

~~bot = 5 e e e~~

$wt = 0$

Profit = $18 + 18 + 10 \times \frac{5}{7}$

$= 36 + \frac{50}{7}$

$= 43$

$S = \{ 0, 1, 1, 5/7 \}$

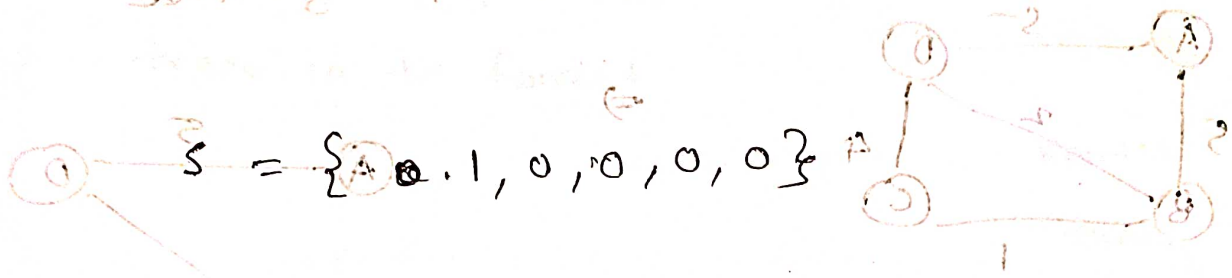
2. $n = 5$ $m = 60$

$\{P_1 P_2 \dots P_5\} = \{30, 20, 100, 90, 160\}$

$\{w_1 \; w_2 \dots w_5\} = \{5, 10, 20, \dots\}$

	1	2	3	4	5
P	30	20	100	90	160
W	5	10	20	30	40
P/W	6	2	5	3	4

(1) (5) (2) (4) (3)

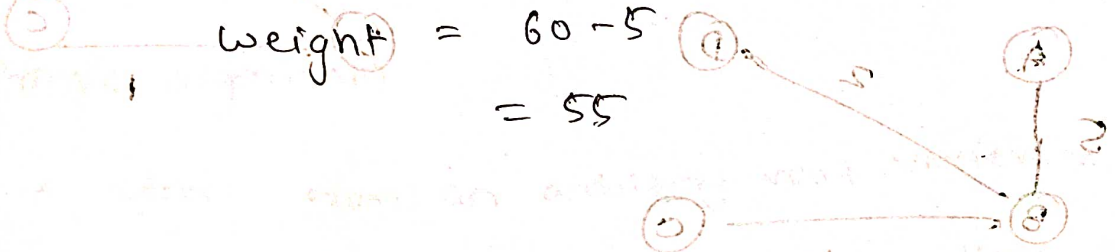


$S = \{1, 0, 0, 0, 0\}$

Profit = 30

Weight = 60 − 5

= 55



$S = \{1, 0, 0, 0, 0\}$

Profit = 30 + 100

weight = 55 − 20

= 35

$S = \{1, 0, \frac{7}{8}, 0, 0\}$

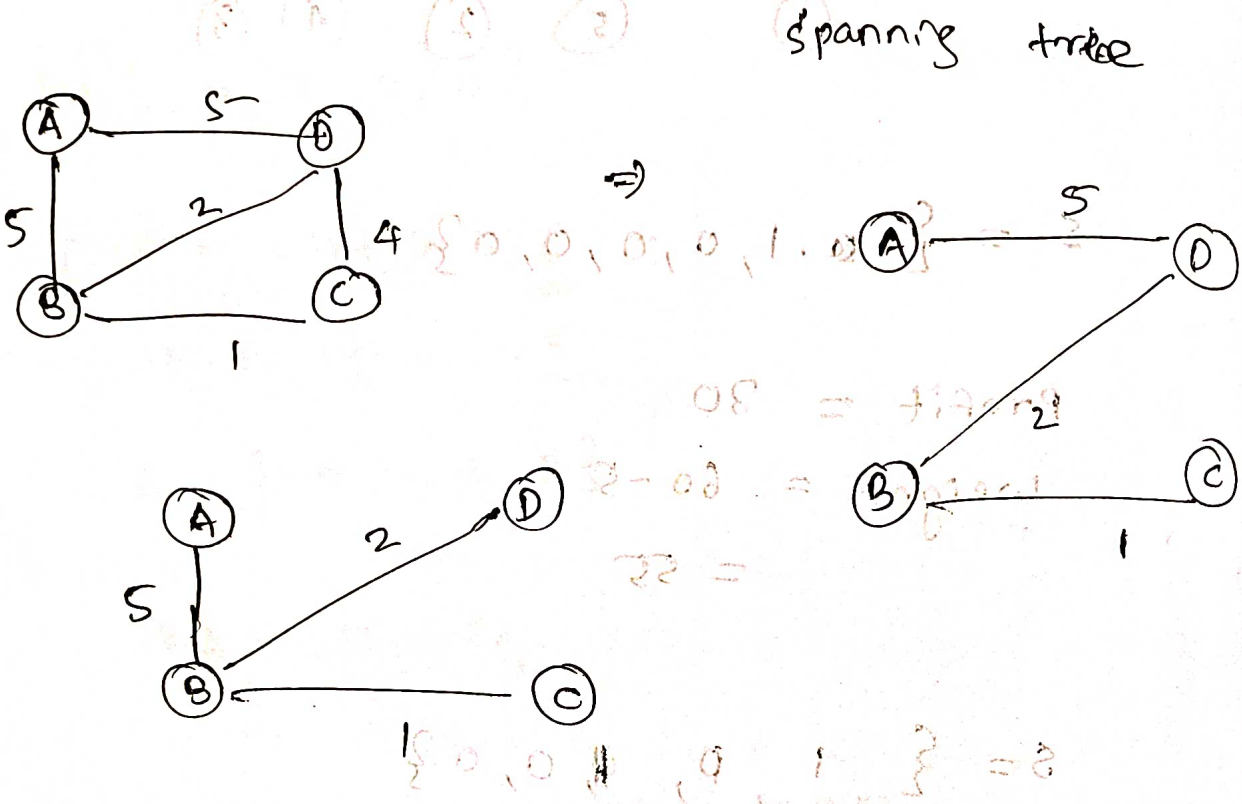
Profit = $30 + 100 + 160 \times \frac{7}{8}$

= 270

Weight = 20

Minimum Spanning tree

Spanning tree : connected acyclic subgraph of a graph G that spans all the vertices.



Generic : MST (G, w)

1. $A = \phi$
2. while A does not form a spanning tree
3. find an edge (u, v) that is safe for A
4. $A = A \cup \{(u, v)\}$
5. return A

Kruskal's

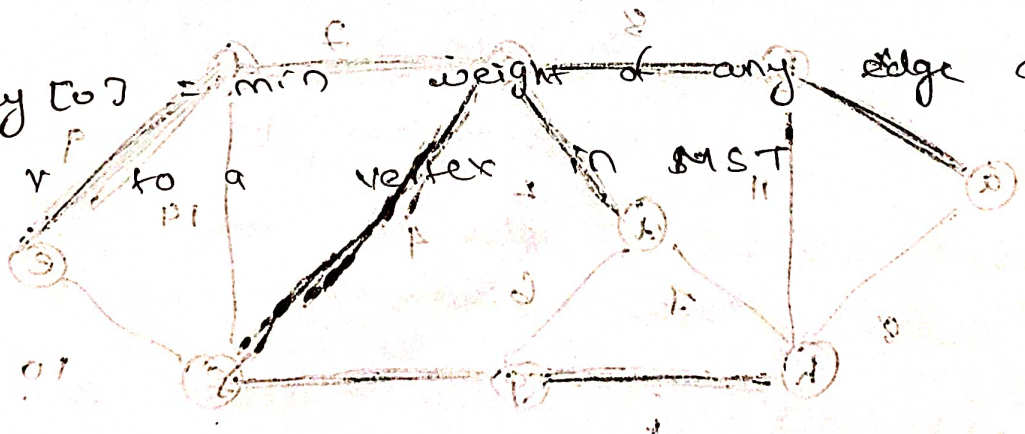
- Set A is a forest
- Start with an edge
- Safe edge added to A is always a least cost edge that connects two distinct trees in the forest

Prim's

- Set A forms a single tree
- Start form a vertex
- Safe edge added to A is always the least cost edge connecting the tree A to a vertex not in the tree

Prim's algorithm

- Start from an arbitary root vertex r
- Uses a priority queue Q to store all vertices not included in MST (V-A)
- Key [v] = min weight of any edge connecting v to a vertex in MST

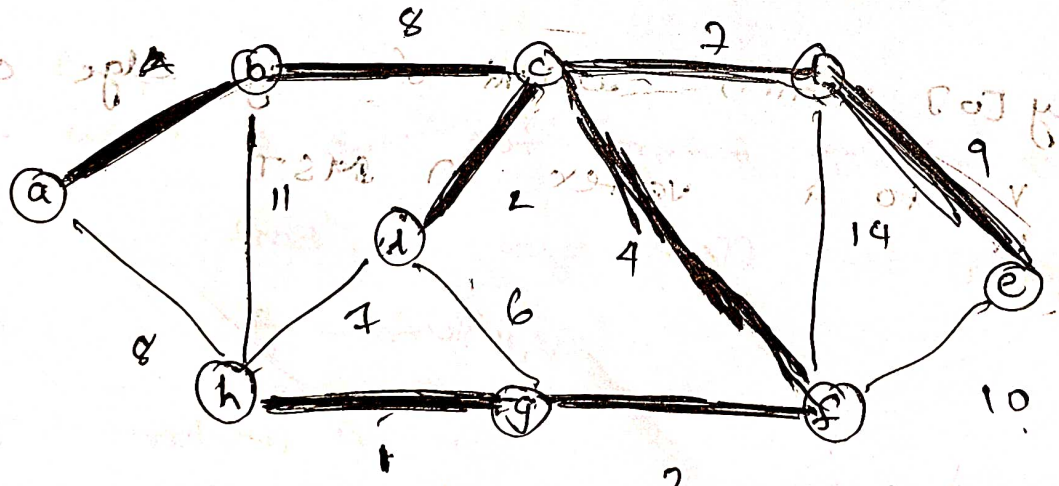


Algorithm

MST - PRIM (G, w, r)

1. for each $u \in G.V$
2. $u.key = \infty$
3. $u.\pi = NIL$
4. $r.key = 0$
5. $Q = G.V$
6. while $Q \neq \emptyset$
7. $u = EXTRACT\text{-}MIN(Q)$
8. for each $v \in G.Adj[u]$
9. if $v \in Q$ and $w(u,v) < v.key$
10. $v.\pi = u$
11. $v.key = w(u,v)$

Q.)



Priority queue Q

4	8	7	9	2	6	8	12
a	b	c	d	e	f	g	h	i

b) total cost = 37

line 6 = |V| times

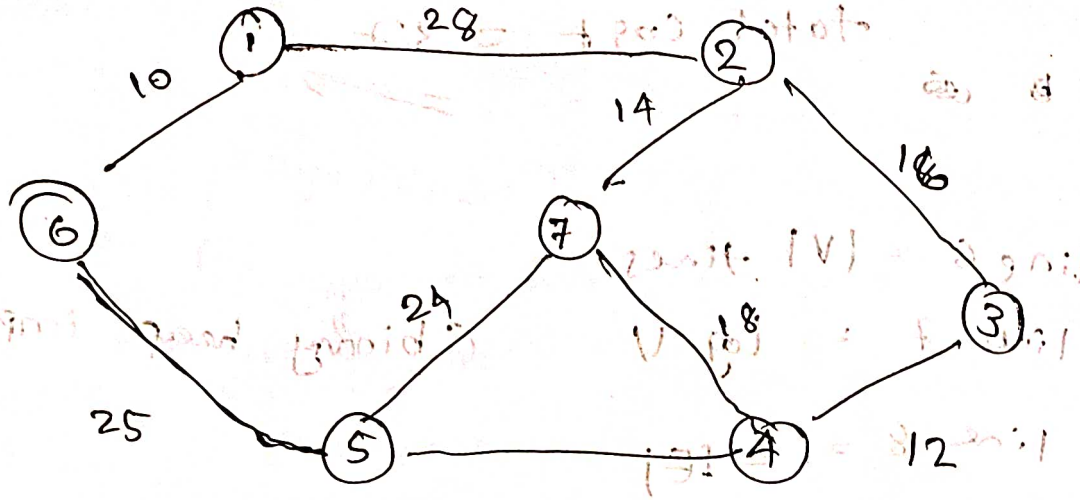
line 7 = log V (binary heap implementation)

line 8 = 2|E|

$O(E \log V)$

Kruskal's Algorithm

```
MST - KRUSKAL (G, w)
1.  A = ∅
2.  for each vertex v ∈ G.V
3.      MAKE-SET (v)
4.  Sort the edges of G.E into increasing order
    by weight w
5.  for each edge (u,v) ∈ G.E taken in
    increasing order by weight
6       if FIND-SET (u) ≠ FIND-SET (v)
7           A = A ∪ {(u, v)}
8           UNION (u, v)
9   Return A
```



22

- make_set

edges

10 ✓

12 ✓

14 ✓

16 ✓

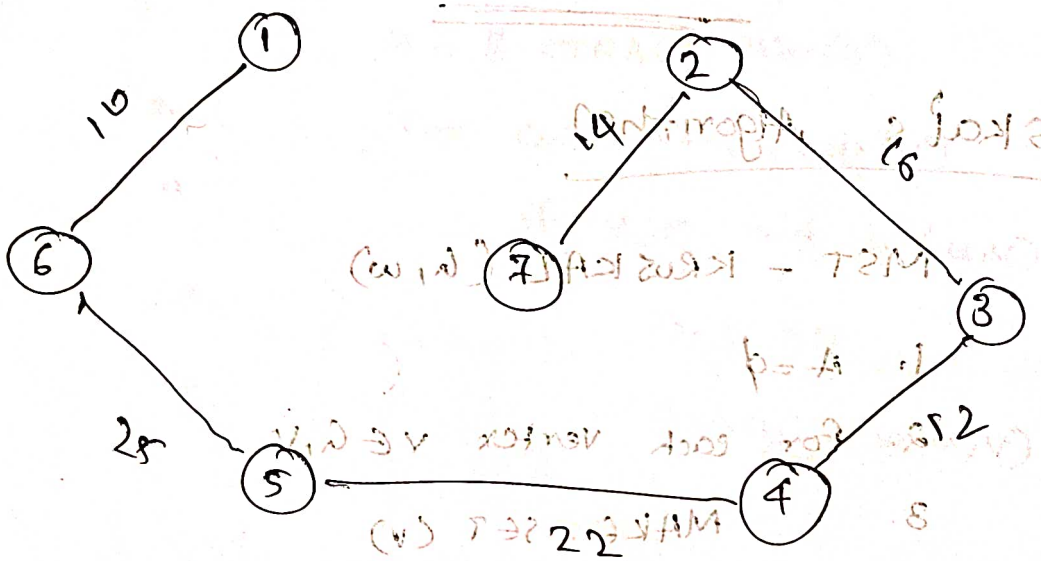
18

22 ✓

24

25 ✓

28



Cost = 99

Complexity :

line 4 = sorting $O(E \log E)$

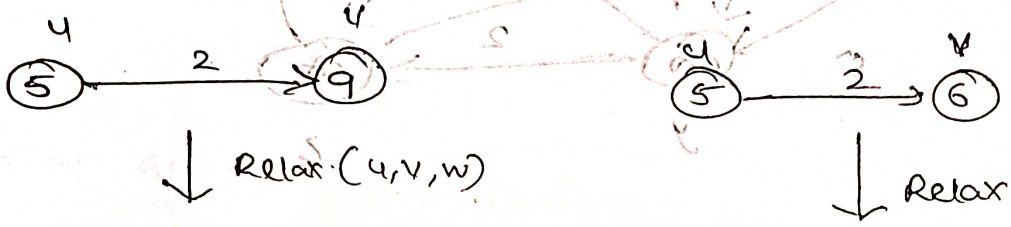
line 5 for loop $O(E)$

$O(E \log E)$

Shortest path

* negative weight cycle in graph ⇒ shortest path not defined

Relaxation

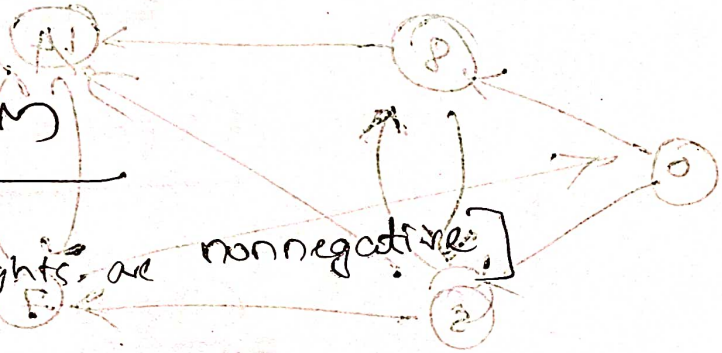


Relax (u, v, w)

IF $d(v) > d(u) + w(u,v)$

$d(v) = d(u) + w(u,v)$

$\pi(v) = u$

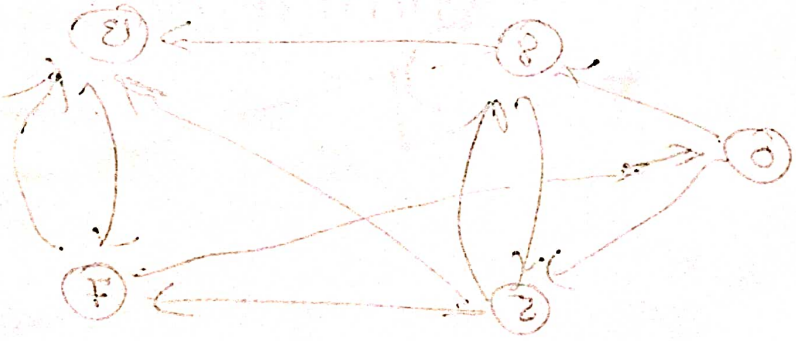


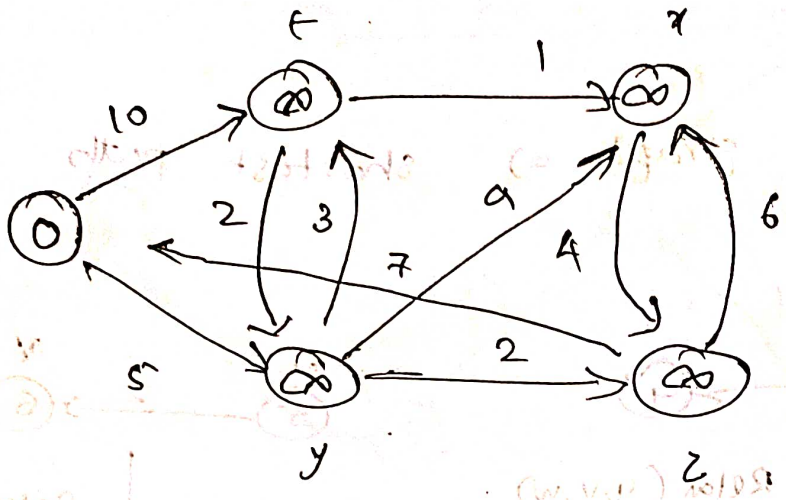
Dijkstra's Algorithm

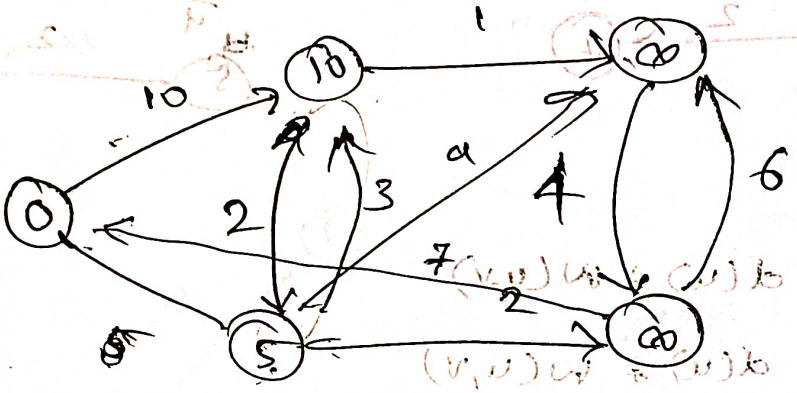
[all edge weights are nonnegative]

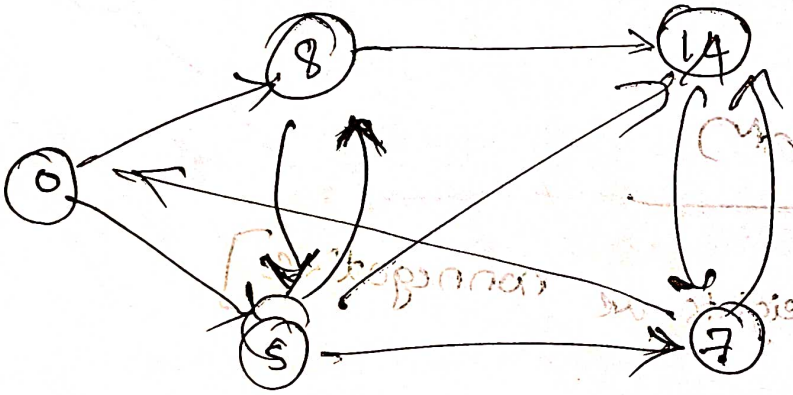
DIJKSTRA (G, w, s)

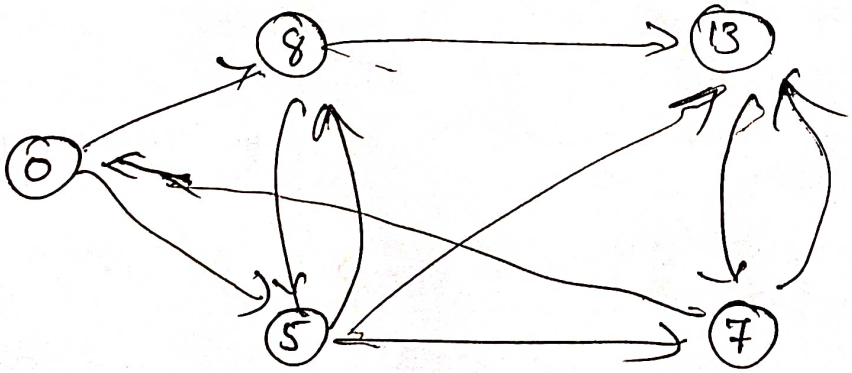
1.

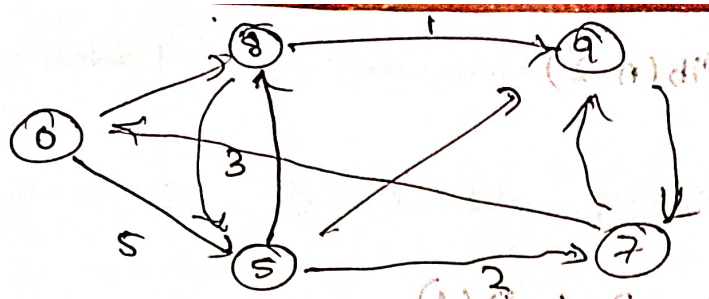



x
1
10
2
3
9
4
6
7
5
2
y


1
10
10
2
3
9
4
6
7
2
5


8
14
0
5
7


8
13
0
5
7



Analysis

Q → priority queue.

Time complexity depends on priority queue.

option -1 Array

$O(V^2)$

option -2 Binary heap

$O((E+V)\log V)$

option 3;

$O(V \log E)$

Dynamic programming

```
fib(n)
{
  if n ≤ 2
    return 1
  else
```